
ANALYSIS OF ACTIVITY IN SECURITY LOGS

By

*Manav Kumar M, **CWID:** 869213850*

*Zarif Shams, **CWID:** 886995554*

AGENDA

- INTRODUCTION
- EXISTING & PROPOSED SYSTEM
- DATASET
- TECHNOLOGIES USED
- DEVELOPMENT FLOW
- DATA PREPROCESSING
- HADOOOP HDFS
- YARN & SPARK CONFIG
- PySpark ANALYSIS & VISUALIZATION
- CONCLUSION

INTRODUCTION

- **What are System Logs?**

System logs are records of events generated by operating systems, applications, and network devices.

They include information like timestamps, user activity, errors, warnings, and system events.

- **Why Analyze Logs?**

To monitor system health.

Detect anomalies and security breaches.

Understand user behavior and system performance.

- **Challenges with Traditional Systems:**

Logs are huge, unstructured, and generated continuously.

Traditional storage/processing tools often fail to scale.

EXISTING & PROPOSED SYSTEM

- **Existing System:** Traditionally, Security logs get ingested by a security tool called a SIEM (Security information and event manager) and those security logs get put into a Hadoop HDFS system for scalable storage & processing. One major drawback of this SIEM + HDFS setup is the lack of security analysis done
- **Proposed System:** The Proposed system is an extension of the existing system where the logs will be put into a HDFS system and by using Spark we can do analysis which involves anomaly score analysis and IP frequency & attack distribution and other analysis.

DATASET

	A	B	C	D	E	F	G	H	I	J	K	L
1	Timestamp	Source IP Address	Destination IP Address	Source Port	Destination Port	Protocol	Packet Length	Packet Type	Traffic Type	Payload Data	Malware Indicators	Anomaly Scores
2	2023-05-30 6:33	103.216.15.12	84.9.164.252	31225	17616	ICMP	503	Data	HTTP	Qui natus odio asperiores nam. Op Maiores possimus ipsum saepe vita	IoC Detected	28.67
3	2020-08-26 7:08	78.199.217.198	66.191.137.154	17245	48166	ICMP	1174	Data	HTTP	Aperiam quos modi officiis veritatis Illo animi mollitia vero voluptates er	IoC Detected	51.5
4	2022-11-13 8:23	63.79.210.48	198.219.82.17	16811	53600	UDP	306	Control	HTTP	Perferendis sapiente vitae soluta. F	IoC Detected	87.42
5	2023-07-02 10:3	163.42.196.10	101.228.192.255	20018	32534	UDP	385	Data	HTTP	Totam maxime beatae expedita explicabo porro labore. Minima ab fugit		15.79
6	2023-07-16 13:1	71.166.185.76	189.243.174.238	6131	26646	TCP	1462	Data	DNS	Odit nesciunt dolorem nisi iste iusto. Animi voluptates soluta quis dolori Quia illo fugit eligendi doloremque. In doloremque autem iure.		0.52
7	2022-10-28 13:1	198.102.5.160	147.190.155.133	17430	52805	UDP	1423	Data	HTTP	Repellat quas illum harum fugit incidunt exercitationem illum. Voluptate		5.76
8	2022-05-16 17:5	97.253.103.59	77.16.101.53	26562	17416	TCP	379	Data	DNS	Qui numquam inventore repellat ratione fugit odit. Quidem est possimu Excepturi quisquam iusto provident adipisci.		31.55
9	2023-02-12 7:13	11.48.99.245	178.157.14.116	34489	20396	ICMP	1022	Data	DNS	Amet libero optio quidem praesentii Eveniet dolor odio libero. Minus iste	IoC Detected	54.05
10	2023-06-27 11:0	49.32.208.167	72.202.237.9	56296	20857	TCP	1281	Control	FTP	Veritatis nihil amet atque molestias	IoC Detected	56.34
11	2021-08-15 22:2	114.109.149.113	160.88.194.172	37918	50039	UDP	224	Data	HTTP	Consequatur ipsum autem reprehenderit quae. Doloribus dicta laborios		16.51
12	2022-07-20 13:2	177.21.83.200	196.218.124.169	35538	35006	ICMP	661	Data	HTTP	Sequi maxime voluptate ea. Eius officiis eaque. Nesciunt aspernatur re Eius dolore molestias. Voluptas laboriosam vero suscipit assumenda in		24.91
13	2022-06-26 15:1	92.4.25.171	112.43.185.24	10903	36817	TCP	281	Control	HTTP	Nihil praesentium asperiores omnis ullam libero. Facilis eius aspernatu		86.07
14	2020-09-30 21:3	57.91.207.84	98.96.110.38	53471	38048	ICMP	64	Control	DNS	Earum sit est et eaque ipsam. Vero repellendus error iusto eveniet.		74.2
15	2021-01-13 2:30	80.28.21.123	111.204.103.106	43729	10845	ICMP	1341	Data	HTTP	Ab voluptatibus nam aperiam. Est sed nostrum animi quae quas. Rerur Cupiditate provident doloremque aliquam maiores tenetur tempora. Similique dolor ullam illo placeat.		62.14
16	2023-02-01 13:1	54.163.130.178	62.112.149.214	47795	35243	UDP	913	Data	DNS	Sit doloremque explicabo. Cupiditate placeat exercitationem. Dolores n Suscipit itaque inventore eveniet ipsa. Ea ipsa minima impedit aliquam		72.65
17	2023-09-15 20:3	130.163.192.252	12.192.2.112	24912	21176	TCP	425	Data	DNS	Voluptates nam doloremque. Eius amet officiis aperiam delectus molliti		91.56
18	2023-04-18 16:4	4.255.187.165	136.159.186.239	39934	5259	TCP	838	Control	HTTP	Sed facere culpa aliquam. Adipisci error quisquam reprehenderit itaque		43.83

TECHNOLOGIES USED

1) Hadoop (HDFS):

Hadoop Distributed File System (HDFS) is the storage layer of the Apache Hadoop framework. It allows large files to be stored across multiple nodes in a cluster, ensuring fault tolerance and high throughput for data access. HDFS is designed to handle massive amounts of data efficiently.

2) Apache Spark:

Apache Spark is a fast, in-memory distributed computing framework that supports large-scale data processing. It improves upon Hadoop's MapReduce by offering faster execution, support for various languages (Python, Java, Scala), and libraries for SQL, streaming, machine learning, and graph processing.

3) Python (PySpark):

PySpark is the Python API for Apache Spark, allowing users to harness Spark's power using Python. It enables writing Spark applications using Python, making it easier to work with RDDs, DataFrames, and Spark SQL, while leveraging Python's rich ecosystem of libraries.

TECHNOLOGIES USED

4) **Log Datasets from Kaggle:**

Kaggle is a popular platform for data science competitions and datasets. In this project, publicly available **log datasets** from Kaggle were used to simulate real-world scenarios, such as user activity logs, server access logs, or system event logs.

5) **Streamlit:**

Streamlit is a Python-based open-source framework for building and deploying interactive web apps for data science and machine learning projects. It allows you to create dashboards and visualizations directly from Python code with minimal effort.

Development Flow

Hadoop HDFS Creation

YARN Configuration

Spark Configuration

Import Dataset

PySpark Analysis

Streamlit Frontend Visualization

DATA PREPROCESSING

```
2025-05-01 12:30:22 ERROR [auth] Failed login for user admin from IP 192.168.1.10
```

- **PREPROCESSING STEPS:**

Extract fields: Timestamp, Log Level, Source Module, Message

Convert formats: CSV or JSON for structure

Tokenization & filtering irrelevant data

Store intermediate outputs in HDFS

Hadoop HDFS

Configure Hadoop
Add dataset into HDFS

/

Go!

Show 25 entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
☐	-rw-r--r--	zarif	supergroup	17.05 MB	Apr 28 20:00	1	128 MB	cyberlogs.csv
☐	drwxr-xr-x	zarif	supergroup	0 B	Apr 28 20:14	0	0 B	user

Showing 1 to 2 of 2 entries

Previous

1

YARN and Spark Config

Configure YARN and Spark



▼ Cluster

[About](#)
[Nodes](#)
[Node Labels](#)
[Applications](#)
NEW
NEW_SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
[Scheduler](#)

► Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources
0	0	0	0	0	<memory:0 B, vCores:0>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes
1	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluste
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>	0

Dump scheduler logs 1 min ▼

Application Queues

Legend: Capacity Used Used (over capacity) Max Capacity Users Requesting Resources Auto Created Queues

▼ root

- + root.prod
- + root.dev

Show 20 ▼ entries

ID ▼	User ▼	Name ▼	Application Type ▼	Application Tags ▼	Queue ▼	Application Priority ▼	StartTime ▼	LaunchTime ▼	FinishTime ▼	State ▼	FinalStatus ▼	Running Contain
------	--------	--------	--------------------	--------------------	---------	------------------------	-------------	--------------	--------------	---------	---------------	-----------------

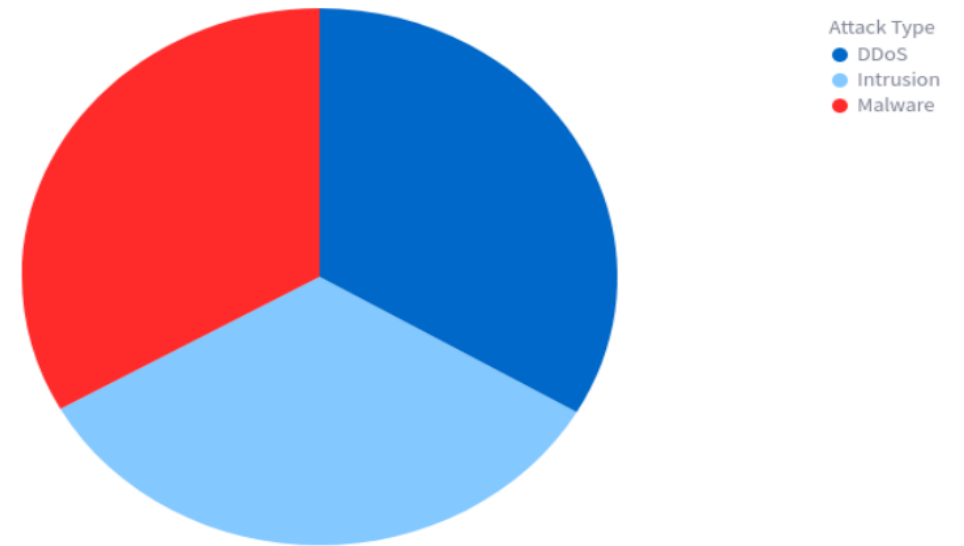
PySpark Analysis and Visualization

1. Top 10 Attack Types

Filter to valid attack types and remove nulls

```
filtered_attack_types_df = (  
    df.filter(  
        col("Attack Type").isNotNull() &  
        col("Attack Type").isin("Malware", "DDoS", "Intrusion")  
    )  
    .groupBy("Attack Type")  
    .agg(count("*").alias("Count"))  
)
```

Top Attack Types



PySpark Analysis and Visualization

2. Attack Frequency Over Time

```
attack_freq_monthly_df = (  
    df  
    .filter(col("Timestamp").isNotNull())  
    .withColumn("Timestamp", to_timestamp("Timestamp"))  
    .withColumn("Month", date_format("Timestamp", "yyyy-MM"))  
    .groupBy("Month")  
    .agg(count("*").alias("Event Count"))  
    .orderBy("Month")  
)
```

Full
17

Attack Frequency Per Month

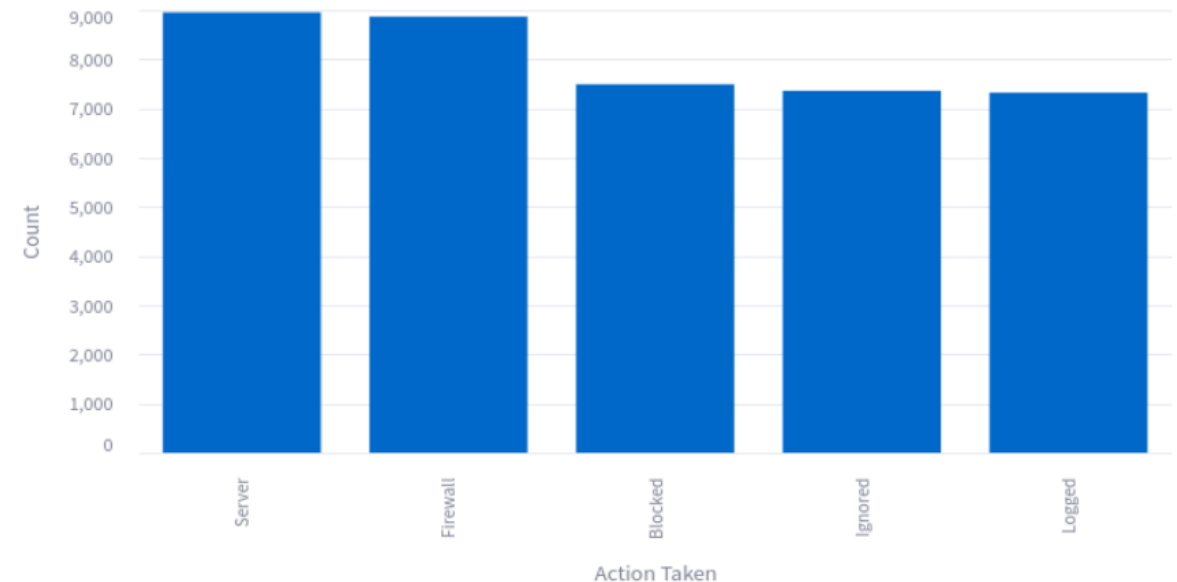


PySpark Analysis and Visualization

3. Action Taken Distribution

```
# Remove nulls and count action types
action_taken_df = (
    df.filter(col("Action Taken").isNotNull())
      .groupBy("Action Taken")
      .agg(count("*").alias("Count"))
      .orderBy("Count", ascending=False)
)
```

 Actions Taken by Security System



PySpark Analysis and Visualization

4. Anomaly Score Distribution

```
anomaly_monthly_df = (  
    df  
    .filter(col("Anomaly Scores").isNotNull() & col("Timestamp").isNotNull())  
    .withColumn("Anomaly Scores", col("Anomaly Scores").cast("float"))  
    .withColumn("Timestamp", to_timestamp("Timestamp"))  
    .withColumn("Month", date_format("Timestamp", "yyyy-MM"))  
    .groupBy("Month")  
    .agg(avg("Anomaly Scores").alias("Average Anomaly Score"))  
    .orderBy("Month")  
)
```



CHALLENGES

- **Unstructured Data:**

- Log formats differ across sources

- Parsing and standardizing required effort

- **Volume of Data:**

- Logs generated every second

- Needed to optimize for storage and performance

- **Real-time Processing:**

- Hadoop is batch-oriented; hard to catch real-time issues without extra tools (Kafka, Spark Streaming)

- **Tool Integration:**

- Combining Hadoop tools (e.g. HDFS) with visual dashboards was complex

CONCLUSION & FUTURE SCOPE

- **CONCLUSION:**

Successfully stored and analyzed system logs using HDFS

Extracted useful patterns and visualized system behavior

HDFS + Spark proved scalable and efficient

- **FUTURE SCOPE:**

Integrate with **Apache Kafka** for real-time streaming

Use **ML algorithms** for intelligent anomaly detection

Develop **dashboards**

Automate alerts for specific log triggers (e.g., too many failed logins)